

MonteQ 论文总结

论文: MonteQ: A Monte Carlo Tree Search Based Quantum Circuit Synthesis Framework

作者: Mulundano Machiya, Matt Menickelly, Paul Hovland, Ji Liu

主题: 基于 Monte Carlo Tree Search 的哈密顿量模拟量子线路综合框架

1. 研究背景与问题定义

哈密顿量模拟是实现量子优势的重要方向之一。实际量子模拟中的目标算符通常可以近似表示为一串 Pauli 旋转:

$$U \approx \prod_j e^{-i\theta_j P_j}$$

其中, P_j 是 Pauli string。为了在量子硬件上执行, 需要把这些 Pauli 旋转综合为 Clifford + R_z 门集。由于两量子比特门, 尤其是 CNOT 门, 通常比单量子比特门错误率更高, 因此本文的主要优化目标是:

在保持或近似保持目标量子操作的前提下, 尽可能减少综合线路中的 CNOT 门数。

传统方法多关注局部重写、门抵消或贪心式 Pauli string 排序。MonteQ 的核心出发点是: Pauli string 的实现顺序本身会显著影响最终线路的 CNOT 数, 而可选顺序空间是阶乘级的, 不能简单穷举。

2. 基础算法

2.1 Clifford + R_z 综合基础

任意 Pauli 旋转可以通过 Clifford 电路转化为单量子比特 R_z 旋转:

$$e^{-i\theta P_2} = CL_n e^{-i\theta P_1} CL_n^\dagger$$

也就是说, 可以先用 Clifford 把某个 Pauli string 映射到 weight 为 1 的 Pauli string, 再施加 R_z , 最后处理尾部 Clifford。

在实现一个 Pauli rotation 后, 尾部 Clifford 会被交换到线路后方。这个操作会改变后续尚未实现的 Pauli string, 因此:

- 不同 Pauli string 实现顺序会产生不同的中间 Pauli word;
- 不同中间 Pauli word 会影响后续 Clifford 综合成本;
- 最终 CNOT 数强烈依赖高层调度顺序。

2.2 Pauli string 的向量化表示

论文采用 2-bit 编码表示 Pauli 算符:

Pauli	编码
I	[00]
X	[10]

Pauli	编码
Y	[11]
Z	[01]

一个 n 比特 Pauli string 被表示为长度 $2n+1$ 的向量，其中前 n 位是 X-side，中间 n 位是 Z-side，最后一位表示符号。多个 Pauli string 构成一个 Pauli word 矩阵。

这种表示便于对 Hadamard、Phase 和 CNOT 共轭操作进行向量化更新。

2.3 MDP 建模

MonteQ 把量子线路综合问题建模为 Markov Decision Process, MDP。

MDP 元素	MonteQ 中的含义
状态 s	当前 Pauli word, 以及已经执行的实现序列
动作 a	选择下一个要实现的 Pauli string
状态转移 $\Phi(s, a)$	实现所选 Pauli string 后得到新的 Pauli word
奖励 $r(s, a)$	负的 CNOT 数, 即 CNOT 越少奖励越高
终止状态	所有 Pauli string 都已经被综合为 Clifford + R_z 形式

因为每一步综合和 Pauli word 更新是确定性的，所以该 MDP 是确定性序贯优化问题。

2.4 Monte Carlo Tree Search, MCTS

MonteQ 使用 MCTS 在巨大的 Pauli string 排序树中搜索低 CNOT 解。每轮 MCTS 包含四步：

1. Selection
从根节点出发，使用 UCT 策略选择子节点，平衡利用与探索。
2. Expansion
如果当前叶节点不是终止状态，则扩展一个新的子节点。
3. Simulation / Rollout
从新节点开始，用简单贪心策略快速走到终止状态。论文实现中，rollout 每步选择 Pauli weight 最小的 Pauli string。
4. Backpropagation
将终止线路的奖励，即 CNOT 数负值，回传到路径上的节点，更新节点价值估计。

其选择策略基于 UCT：

$$a^* = \arg \max_{a \in A} \left[r(s, a) + V(\Phi(s, a)) + \mu \sqrt{\frac{\log N_s}{N_{s,a}}} \right]$$

其中：

- $V(\Phi(s, a))$ 是子状态价值估计；
- N_s 是状态 s 被模拟次数；

- $N_{s,a}$ 是动作 a 被选择次数;
- μ 是探索系数, 论文中设为 $\sqrt{2}$ 。

3. MonteQ 的改进方法

3.1 两层框架设计

MonteQ 采用两层设计:

层级	功能
高层 MCTS	搜索 Pauli string 的实现顺序
低层 heuristic	给定一个 Pauli string, 生成具体 Clifford + R_z 综合线路

这种设计使 MonteQ 不依赖单一综合策略。只要低层 heuristic 能把 Pauli word 从长度 K 变成 $K - m$, 并返回对应 Clifford + R_z 电路, 就可以嵌入 MonteQ。

3.2 支持 unitary preserving 与 unitary modifying

MonteQ 支持两种 Pauli string 排序模式。

3.2.1 Unitary Preserving 模式 该模式保持目标么正算符不变。论文用有向无环图 DAG 表示 Pauli string 之间的反对易关系:

- Pauli string 是图节点;
- 如果两个 Pauli string 反对易, 则从原序列中靠前者指向靠后者;
- 入度为 0 的节点构成 front layer;
- 每次只能从 front layer 中选择要实现的 Pauli string。

这样既允许可交换项重排, 又保证反对易项的相对顺序不被破坏。

3.2.2 Unitary Modifying 模式 该模式把 Pauli string 近似视为全部可交换, 因此允许任意重排。这样会放松原 CNOT 数。

3.3 逻辑层 greedy heuristic

在全连接量子比特假设下, MonteQ 设计了逻辑层贪心 heuristic。

其核心目标是降低当前 Pauli string 的 Pauli weight。CNOT 共轭可以把某些 Pauli 对变成 identity, 从而减少 Pauli weight。论文定义:

- Reducible pairs, RP: 经过 CNOT 共轭后能降低 Pauli weight 的 Pauli 对;
- Increasing pairs, IP: 经过 CNOT 共轭后会增加 Pauli weight 的 Pauli 对。

对候选 Clifford block, 定义收益:

$$benefit_{ij} = \#RP_{ij} - \#IP_{ij}$$

该 heuristic 会选择 benefit 最大的 Clifford block，并重复执行，直到当前 Pauli string 被降到 weight 1，再用单量子比特门转成 Z 方向并接入 R_z 。

相比只看当前 Pauli string 的简单贪心策略，这个方法还考虑了候选操作对整个 Pauli word 中其他 Pauli string 的影响。

3.4 硬件感知 heuristic

真实量子硬件通常不是全连接结构，因此 MonteQ 还提供 hardware-aware heuristic。

该方法使用：

$$\Omega_K = [X|Z|S]$$

表示 Pauli word，并构造 occupancy matrix：

$$O = X||Z$$

O 表示哪些量子比特上存在非 identity Pauli 项。结合硬件耦合图 C 和距离矩阵 D ，论文定义距离指标：

$$dist = \sum_n [(O \times D) \cdot * O]_{kn}$$

硬件感知 heuristic 的目标是优先消除距离较远、可能引入更多 SWAP 成本的非 identity Pauli 项。如果 heuristic 陷入循环，则回退到之前状态并切换到更激进的 greedy step，确保最终能完成综合。

3.5 工程优化

MonteQ 还加入了若干实用优化：

- 使用 Qiskit 框架实现线路表示；
- 对尾部 Clifford U_{CL} 使用 Qiskit 的 GreedySynthesisClifford pass 进行优化；
- 即使 MCTS expansion 没有直接到达终止节点，也保存 rollout 产生的完整可行解；
- 多个候选线路 CNOT 数相同时，用线路深度作为 tie-breaker；
- 用户可以通过调整 MCTS iteration 数量，在运行时间和 CNOT 优化质量之间折中。

4. 核心实验结果

论文使用的 benchmark 包括：

- Fermi-Hubbard 模型；
- Heisenberg 模型；
- 分子哈密顿量：LiH、H₂O、NH₃；
- UCC ansatz。

主要评价指标是 CNOT gate count。

4.1 逻辑层、Unitary Preserving、单次 MCTS iteration

在单次 iteration 设置下, MonteQ 已经表现出较强竞争力。

对比对象	结果
Qiskit-Rustiq	18 个 benchmark 中赢 17 个; 最大 CNOT 降低 51.6%, 平均降低 23.5%
QuCLEAR	18 个 benchmark 中赢 13 个; 最大 CNOT 降低 68.5%, 平均降低 16.9%
编译时间	比 QuCLEAR 快, 但通常比 Rustiq 慢

这说明即使不充分利用 MCTS 多轮搜索, MonteQ 的 rollout policy 与逻辑层 heuristic 也已经有效。

4.2 多次 MCTS iteration

当 iteration 数增加到 200 时, MonteQ 相比自身单次 iteration 进一步优化:

指标	结果
平均 CNOT 降低	10.3%
最大 CNOT 降低	30.8%
整体效果	200 iterations 后完全优于 Qiskit-Rustiq

论文还指出, 运行时间与 iteration 数大致呈线性关系, 因此 MonteQ 可以通过 iteration 数量控制时间-质量折中。

4.3 Unitary Modifying 结果

在允许修改么正约束的设置下, MonteQ 与 Qiskit-Rustiq 的 unitary modifying 优化比较:

指标	结果
胜出数量	18 个 benchmark 中赢 17 个
最大 CNOT 降低	60.2%
平均 CNOT 降低	27.2%

这说明 MonteQ 在放松排序约束后可以进一步利用 Pauli string 的共同化简机会。

4.4 Hardware-aware 结果

在 IBM FakeManhattanV2 heavy-hex 架构上, MonteQ 与 Tetriss、TKet 和 Qiskit transpiler 比较:

对比对象	结果
Tetris	10 个 benchmark 中赢 6 个；最大降低 21%，平均降低 2.42%
TKet	10 个 benchmark 全部胜出；最大降低 56.5%，平均降低 24.8%
Qiskit transpiler	10 个 benchmark 全部胜出；最大降低 42.5%，平均降低 26.8%

论文指出，hardware-aware heuristic 在分子哈密顿量和 UCC ansatz 这类 Pauli string 较稠密的任务上表现更好；而在部分较稀疏的动态模型中，Tetris 可能更优。

5. 核心结论与成果

5.1 核心结论

1. Pauli string 的高层实现顺序是 CNOT 优化的重要因素。
单纯局部重写或固定贪心排序不足以充分探索全局优化空间。
2. 将线路综合建模为 MDP 是可行的。
状态、动作、转移和奖励都能自然对应到 Pauli rotation 综合过程。
3. MCTS 能有效采样阶乘级搜索空间。
它不保证全局最优，但能在有限 iteration 内找到低 CNOT 线路。
4. 两层设计提升了框架灵活性。
高层 MCTS 与低层 heuristic 解耦，使 MonteQ 能支持不同优化目标，包括逻辑层优化、硬件感知优化、unitary preserving 和 unitary modifying。
5. 实验结果表明 MonteQ 在多个场景下优于现有编译器。
尤其在逻辑层全连接设置下，相比 Rustiq 的 CNOT 优化最显著；在硬件感知设置下，也稳定优于 TKet 和 Qiskit transpiler。

5.2 主要成果

- 提出一个基于 MCTS 的量子线路综合框架 MonteQ；
- 将 Hamiltonian simulation circuit synthesis 形式化为 MDP；
- 支持 unitary preserving 与 unitary modifying 两种排序约束；
- 支持逻辑层与硬件感知两类低层 heuristic；
- 在代表性 benchmark 上，相比 Rustiq 等方法实现最高约 53%、平均约 30% 的 CNOT gate count 降低；
- 提供可调节的运行时间与综合质量折中机制。

6. 局限性与未来方向

论文也指出 MonteQ 存在一些限制：

1. 搜索空间仍然是阶乘级。
MCTS 只是采样搜索，不能完全消除大规模 Pauli word 的复杂度问题。
2. 运行时间随 iteration 增长。
虽然近似线性，但大系统上仍存在时间压力。
3. 性能依赖低层 heuristic 质量。
简单 heuristic 已经有效，但在部分稀疏 Pauli string 和硬件受限场景中仍可能不如专门方法。

未来方向包括：

- 按 Pauli weight 截断搜索空间；
- 设计不同优化等级；
- 引入随机截断以改善探索多样性；
- 将 MonteQ 扩展到可表示为 Pauli rotation 序列的一般量子线路。